

Requirement 3 & 4 — Exam Presentation Report

Non-Stationary Multi-Campaign Bidding: Environment, Agents, Benchmarks, and a Critical Reading of Every Result

Online Learning Applications Project

Summary. This report walks through Requirement 3 and Requirement 4 end to end: the shared environment class and how it is parameterised differently for each requirement, the agents used, the three regret benchmarks reported for Requirement 4 and why a single benchmark is not enough, and a detailed, critical commentary on every plot produced. Particular attention is given to the results that were *not* expected going in — the primal-dual agent’s budget under-utilisation, the failure of a naive hyperparameter fix, and a theoretical account of why this is a structural limitation, not a tuning mistake. Nothing is assumed to be self-explanatory; every figure is explained from first principles.

Contents

1	Setting and Project Requirements	3
2	The Shared Environment: One Class, Two Parameterisations	3
2.1	What is common to both modes	3
2.2	mode='drift' — Requirement 3	4
2.3	mode='shocks' — Requirement 4	4
2.4	A known limitation, shared by both, worth flagging explicitly	4
2.5	Requirement 4’s own addition: <code>shock_blocks</code> and <code>piecewise_win_probabilities()</code>	4
2.6	Summary table	4
3	Requirement 3: Recap of the Agent and Results	5
3.1	The agent: Primal-Dual (Hedge + OGD)	5
3.2	Benchmark used for Requirement 3	5
3.3	Results: two experiments, same agent, unchanged	5
4	Requirement 4: Agents and Benchmark Methodology	7
4.1	The three agents compared	7
4.2	Why three benchmarks, not one	7
5	Requirement 4: Results, Image by Image	8
6	The Unexpected Result: Retuning Primal-Dual for shocks	12
6.1	Tuning methodology: three rounds, one hard discovery	12
6.2	Full-scale comparison, four agents	12

7	Why This Happens: A Compact Theoretical Account	14
7.1	Static regret vs. dynamic (tracking) regret – the distinction that explains everything above	14
7.2	Why Sliding-Window / CUSUM do not have this problem	14
7.3	Why raising <code>hedge_eta</code> only partially helps, and cannot be pushed further	14
8	Is This What the Professor Expects? A Critical, Evidence-Based Answer	15
9	Summary Table for the Presentation	15

1 Setting and Project Requirements

The course project asks for online learning algorithms that bid on N advertising campaigns under a shared budget B over T rounds, with a conflict graph forbidding certain campaign pairs from being active in the same round. Requirements 1 and 2 build this machinery for a *stochastic* environment (fixed, unknown win-probability distribution). Requirements 3 and 4 both ask for a *non-stationary* environment, but with two different notions of “non-stationary”:

- **Requirement 3** (p.15 of the project spec): *“Build a highly non-stationary environment... a non-stochastic sequence of highest competing bids... sampled from a distribution that changes quickly over time.”* The algorithm required is a primal-dual method, with the explicit hint: *“Design a primal regret minimizer for the specific problem under study.”*
- **Requirement 4** (p.18): *“Build a slightly non-stationary environment... rounds are partitioned in intervals, in each interval the distribution... is fixed, each interval has a different distribution.”* The algorithm required extends Requirement 2’s Combinatorial-UCB with a sliding window and with a change detector, and asks to **compare** these two against “the primal-dual method” — i.e., the very same agent built for Requirement 3, not a new one.

This last point is easy to miss and turns out to be central to everything that follows: Requirement 4 does not ask for a primal-dual algorithm redesigned for piecewise-stationary environments. It asks to take the Requirement 3 agent *as is* and see how it fares. Section 7 explains why this is very likely deliberate.

2 The Shared Environment: One Class, Two Parameterisations

Requirement 3 and Requirement 4 do not use two different environment classes. Both are built on `AdversarialMultiCampaignEnv` (`utils/environments.py`), selected via a single constructor argument, `mode`:

<code>mode='drift'</code>	Requirement 3
<code>mode='shocks'</code>	Requirement 4

Everything else — the constructor, the round-by-round interaction protocol, the conflict-graph resolution rule, the shared budget bookkeeping — is identical code, shared by both requirements. Only the method that builds the (N, T) matrix `self.m` of highest competing bids differs.

2.1 What is common to both modes

- **Bid sets**: for each campaign i , only bids $b \leq v_i$ are admissible (same rule as Requirement 2).
- **round(`bid_indices`)**: a campaign wins if $b_i \geq m_{t,i}$; when two conflicting campaigns both win, the environment keeps the one with higher realised utility and voids the other. The round always returns (`f_t`, `c_t`, `m_t`) — crucially, `m_t` is always revealed, i.e. **full feedback** (project spec p.16), which is what lets the primal-dual agent reconstruct the counterfactual reward of every bid it did *not* play.
- **reset(`seed`)** and **empirical_win_probabilities()**: identical signatures and semantics in both modes.

2.2 mode='drift' — Requirement 3

$$\begin{aligned} \text{angle}(i, t) &= 2\pi \cdot \text{drift_cycles} \cdot \frac{t}{T} + \phi_i, & \bar{m}_{i,t} &= \text{base_mean} + \text{drift_amplitude} \cdot \sin(\text{angle}(i, t)) \\ m_{i,t} &\sim \text{Beta}(\bar{m}_{i,t} \cdot s, (1 - \bar{m}_{i,t}) \cdot s) \end{aligned}$$

With `drift_cycles=10`, `drift_amplitude=0.35`, `base_mean=0.5` over $T = 10000$: the Beta mean oscillates sinusoidally in $[0.15, 0.85]$, completing 10 full cycles (period ≈ 1000 rounds). Each campaign i gets an independent random phase ϕ_i , so the four campaigns are never synchronised. **The distribution changes at every single round** — there is no round at which it is locally constant.

2.3 mode='shocks' — Requirement 4

The horizon is cut into `n_regimes=5` blocks of `block_size=2000` rounds. At init, 5 candidate (α, β) regimes are drawn once (means uniform in $[0.1, 0.9]$). For every block, one of the 5 regimes is picked uniformly at random and $m_{i,t}$ is sampled i.i.d. from it for the whole block:

$$m_{i,t} \sim \text{Beta}(\alpha_b, \beta_b), \quad t \in [b \cdot 2000, (b+1) \cdot 2000), \quad b = 0, \dots, 4$$

The distribution is fixed for 2000 rounds and jumps discontinuously at the block boundary. This is the literal reading of the spec's “each interval has a different distribution.”

2.4 A known limitation, shared by both, worth flagging explicitly

In `_build_shocks`, the regime for block $b+1$ is redrawn *uniformly among all 5*, without excluding the regime just used for block b . There is therefore a $1/5$ chance that two consecutive blocks are actually the *same* distribution, i.e. a block “boundary” that is not a genuine change. This was not corrected (it lives in code shared with Requirement 3, outside this analysis's scope), but it directly explains why the CUSUM detector's reset count (Section 5) does not equal exactly “4 boundaries $\times$ 38 cells”.

2.5 Requirement 4's own addition: `shock_blocks` and `piecewise_win_probabilities()`

While generating the shocks, the environment now also records, for every (campaign, block), the exact (start, end, α, β) used — not an extra random draw, just keeping what was already sampled. The method `piecewise_win_probabilities()` uses this to compute $P(b \geq m)$ *analytically* (Beta CDF) per block, without needing the realised m_t values. This is the ingredient that makes Section 4.2's primary benchmark possible, and it only exists for `mode='shocks'` — there is no analogous notion of “a block's true distribution” when the distribution changes every round.

2.6 Summary table

	Requirement 3 (drift)	Requirement 4 (shocks)
Change granularity	every round	every 2000 rounds (5 blocks)
Governing parameters	<code>drift_cycles=10</code> , <code>amplitude=0.35</code>	<code>block_size=2000</code> , <code>n_regimes=5</code>
Per-campaign sync.	none (random phase)	none (independent regime draw)
Extra metadata	—	<code>shock_blocks</code>
Extra method	—	<code>piecewise_win_probabilities()</code>
Natural benchmark	OPT ^A	Piecewise-expected clairvoyant

3 Requirement 3: Recap of the Agent and Results

3.1 The agent: Primal-Dual (Hedge + OGD)

`PrimalDualMultiCampaignAgent` pairs one Hedge (multiplicative-weights) regret minimiser per campaign, run with full feedback, with a single shared dual variable λ_t updated by online gradient descent (OGD) on the budget constraint. The Lagrangian is

$$L(x, \lambda) = \sum_{i,k} x_{i,k} (f_{t,i,k} - \lambda c_{t,i,k}) + \lambda \rho.$$

With **budget pacing** enabled (the definitive configuration, matching the team’s `giovanni` branch), the dual target is *adaptive*:

$$\rho_t = \frac{\text{remaining budget}}{\text{remaining rounds}} \quad \text{instead of the fixed } \rho = B/T,$$

so the constraint self-adjusts to how much budget is actually left. Final hyperparameters: `hedge_eta` = class default $\sqrt{\log(K_{\max})/T} \approx 0.01517$, `ogd_eta` = 0.017, `budget_pacing` = True.

3.2 Benchmark used for Requirement 3

Requirement 3’s own experiments (`run_req3.py`) benchmark against OPT^A : the best *single fixed* distribution over actions for the whole horizon, computed from `empirical_win_probabilities()`. This is deliberate, not a simplification: Hedge’s regret guarantee is specifically against the best fixed action in hindsight (external/static regret), so OPT^A is the benchmark the algorithm actually has a theorem for.

3.3 Results: two experiments, same agent, unchanged

Experiment	Environment	Final regret (vs OPT^A)	Budget used
A – Stochastic	<code>MultiCampaignEnv</code>	164.08	1581.17/1600 (98.8%)
B – Adversarial	<code>AdversarialMultiCampaignEnv(drift)</code>	253.00	1575.49/1600 (98.5%)

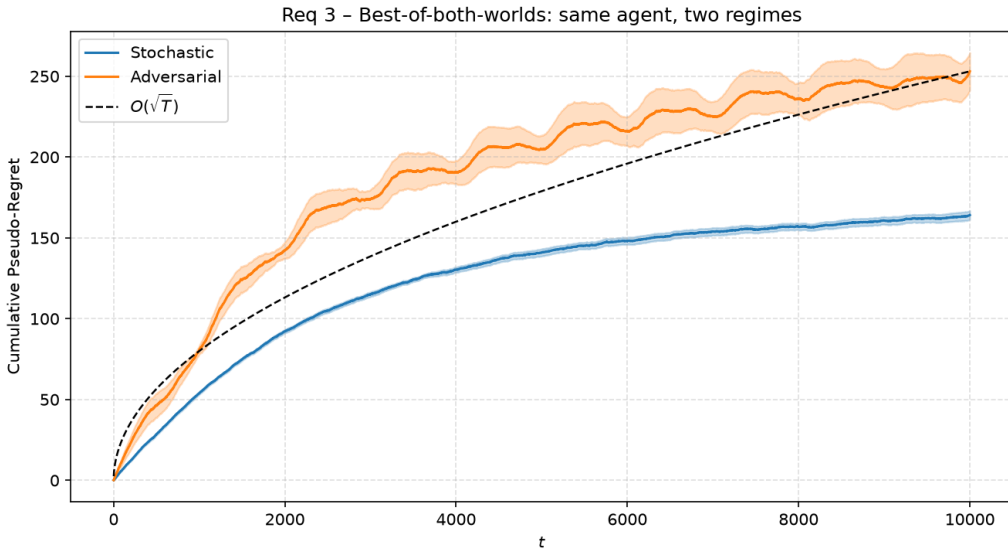


Figure 1: Requirement 3 – cumulative pseudo-regret, same agent, two regimes, with an $O(\sqrt{T})$ reference curve overlaid.

Critical reading. Both curves are visibly concave and both track the $O(\sqrt{T})$ reference reasonably closely – this is the textbook signature of a no-external-regret algorithm actually achieving its guarantee. The adversarial (drift) curve sits consistently above the stochastic one, which is expected: a moving target is intrinsically harder to track even for the best-fixed-in-hindsight comparator itself. Note the small oscillations riding on top of the adversarial curve’s growth (visible as gentle “steps” roughly every ~ 1000 rounds) – these line up with the drift’s own period, and are explained more precisely by Figure 3 below. **This is the clean, textbook-correct behaviour that Requirement 4’s results (Section 5) will be directly contrasted against.**

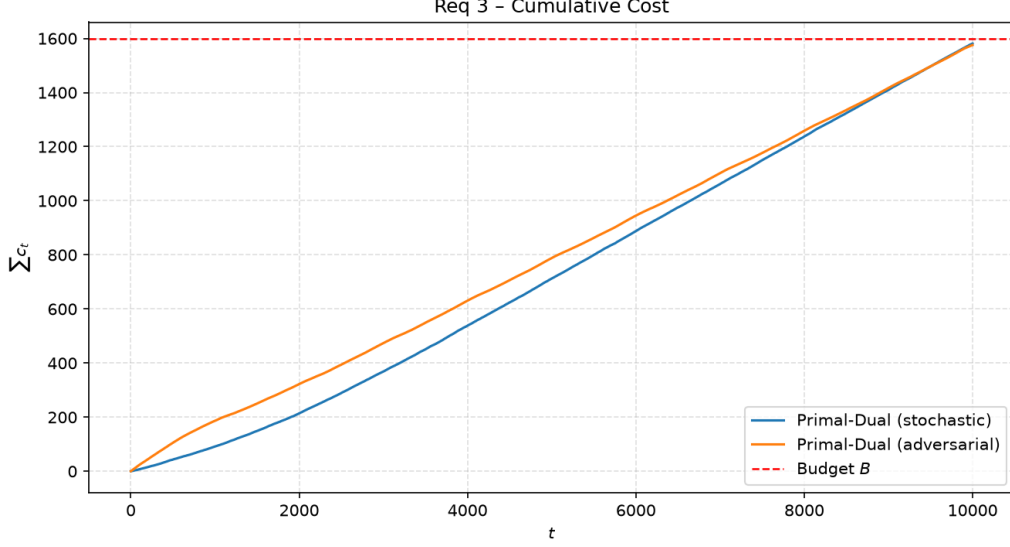


Figure 2: Requirement 3 – cumulative cost vs. the budget line $B = 1600$.

Critical reading. Both curves are almost perfectly straight lines that touch the budget line right at $t = T$, with no early exhaustion and no leftover budget. This is exactly what `budget_pacing` is designed to produce, and here it works as intended – there is nothing more to explain, which is itself the point: this is the baseline of “good” behaviour that Requirement 4’s primal-dual curve (Figure 7) visibly departs from.

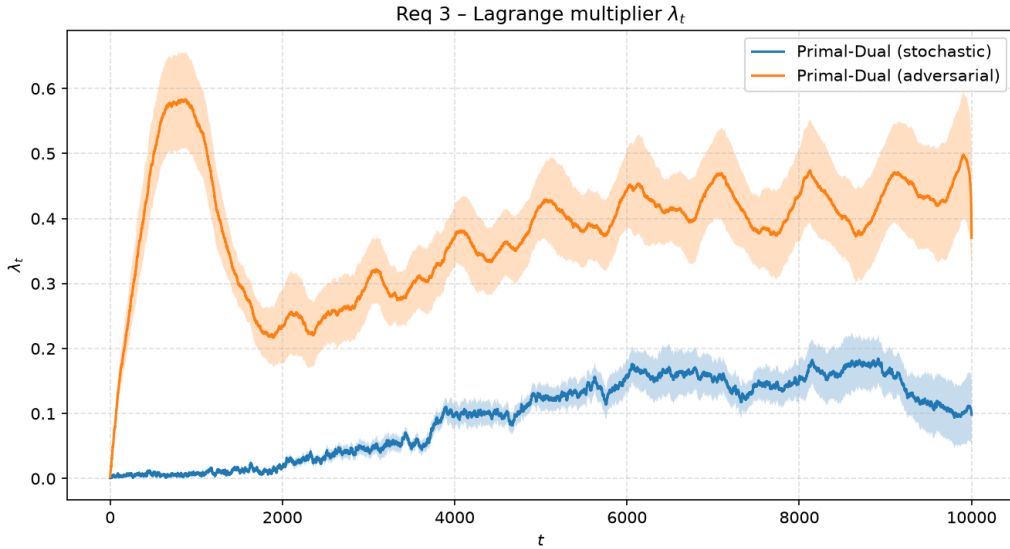


Figure 3: Requirement 3 – Lagrange multiplier λ_t over time.

Critical reading. This is the most informative diagnostic in the whole Requirement 3 analysis. The stochastic λ_t (blue) rises smoothly and settles into a low, stable band ($\approx 0.1\text{--}0.18$). The adversarial λ_t (orange) is far more interesting: after an initial overshoot (the cold start at $\lambda_0 = 0$ causes early over-bidding before the constraint engages), it **oscillates with a period that visibly matches the drift’s own cycle** (≈ 1000 rounds, matching `drift_cycles=10` over $T = 10000$), staying bounded within $[0.2, 0.6]$ for the entire horizon. This is a real-time visual proof that OGD is *tracking* the continuously-moving win-probability distribution smoothly, never getting stuck. **Keep this image in mind – Figure 8 will show the same mechanism failing to do this under shocks.**

4 Requirement 4: Agents and Benchmark Methodology

4.1 The three agents compared

1. **SlidingWindowCombinatorialUCBAgent** – Requirement 2’s `CombinatorialUCBAgent`, restricted to a trailing window of W rounds per (campaign, bid) cell, via a shared deque. $W = \text{block_size} = 2000$ (tied to the interval length, not the textbook $W = 2\sqrt{T} \approx 200$, which would forget under-sampled cells before they are well estimated – confirmed empirically to underperform in earlier tuning).
2. **CUSUMCombinatorialUCBAgent** – same base class, with a per-cell CUSUM change detector (Page, 1954) run on the **win indicator** $w = \mathbf{1}[b \geq m_t]$ (not raw utility, since it is the win *probability* that shifts across blocks, not the campaign’s fixed value v_i). On alarm, only that cell’s statistics reset. $U_T = 4$ (one less than the 5 blocks) feeds the standard formulas for M , h , α .
3. **PrimalDualMultiCampaignAgent** – Section 3’s agent, **reused with the exact same configuration** (`budget_pacing=True`, `ogd_eta=0.017`), only re-run on `mode='shocks'` instead of `mode='drift'`. The project spec names this “the primal-dual method”, not a new one.

Both Combinatorial-UCB variants subclass `CombinatorialUCBAgent` as *fixed in the team’s main branch* (empirical-mean cost in the budget LP, no premature greedy fallback) – this matters because an earlier, divergent copy of the same class (on the `giovanni` branch) had regressed to the original buggy fallback that disables the budget constraint almost every round; reusing that version would have made the whole comparison meaningless.

4.2 Why three benchmarks, not one

This is the single most important methodological decision in the Requirement 4 analysis, and it changed over the course of this work – worth presenting as a deliberate design choice, not a false start.

Dynamic / “prophet” clairvoyant (reference upper bound only).

Solves one LP over the *entire, already-realised* sequence `m_seq` – it knows every future outcome before choosing each round’s action. Even within one stationary block, m_t is still a fresh i.i.d. draw every round; a clairvoyant that knows the specific realised value can always do at least as well as one that only knows the block’s distribution. By Jensen’s inequality,

$$\mathbb{E}[\max_k f_k(m_t)] \geq \max_k \mathbb{E}[f_k(m_t)],$$

so comparing a learner against this oracle inflates regret by a term that is **linear in T regardless of learner quality** – an artefact of the benchmark, not evidence of a learning failure. Kept only as an upper-bound reference (this is exactly what `docs/Req4_Linear_Regret_Baseline.tex` derives in full).

OPT^A (secondary).

Best single *fixed* distribution over actions for the whole horizon (same LP as Requirement 3’s own benchmark, fed `empirical_win_probabilities()`). Kept for continuity with Requirement 3’s own reported numbers, and because it is still Primal-Dual’s own provable-regret benchmark.

Piecewise expected clairvoyant (primary).

Knows the block boundaries and each block’s *true* distribution (analytically, via `piecewise_win_probabilities` but **not** the realised m_t . Solves one LP with one mixed action per block, budget-coupled across blocks. This is the textbook-correct benchmark for a piecewise-stationary environment, and specifically the one Sliding-Window/CUSUM-style algorithms have tracking-regret guarantees against in the literature (e.g. Garivier & Moulines, 2011, for SW-UCB) – unlike OPT^A, which does not reward tracking at all.

5 Requirement 4: Results, Image by Image

Agent	Regret (piecewise, primary)	Regret (OPT ^A)	Regret (prophet, refer
CUSUM Combinatorial-UCB	1443.90	644.55	2717.49
Sliding-Window Combinatorial-UCB	1532.01	732.66	2805.60
Primal-Dual (Req 3 config, unchanged)	1777.40	978.05	3050.99

Agent	Cumulative cost / Budget used
CUSUM Combinatorial-UCB	1598.91/1600 (99.9%)
Sliding-Window Combinatorial-UCB	1595.52/1600 (99.7%)
Primal-Dual (Req 3 config, unchanged)	1193.92/1600 (74.6%)

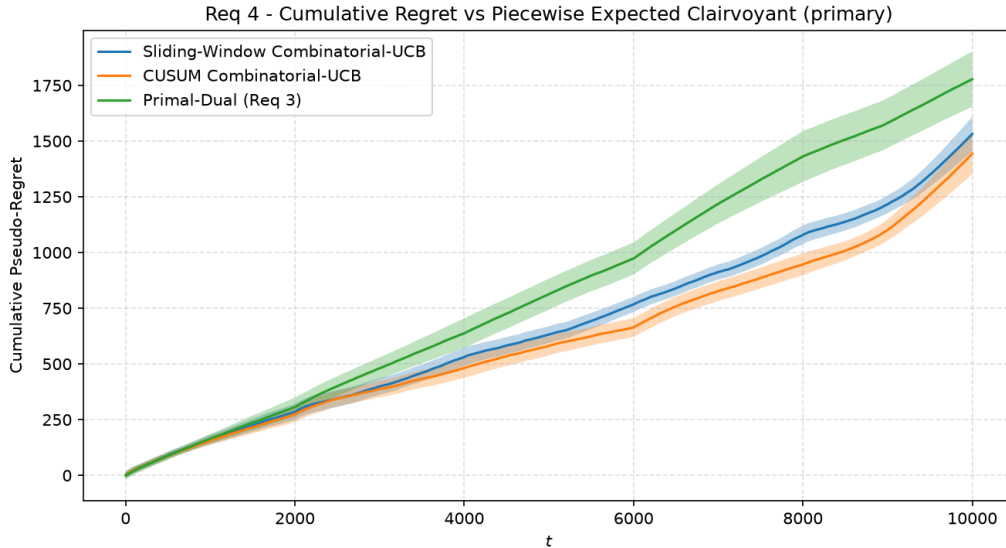


Figure 4: Requirement 4 – cumulative regret vs. the **piecewise expected clairvoyant** (primary diagnostic).

Critical reading. The ranking CUSUM < Sliding-Window < Primal-Dual is unambiguous and holds throughout the horizon, not just at $t = T$. Unlike Requirement 3’s regret curves (Figure 1),

none of these three curves is convincingly concave – they are close to linear, with a visible acceleration in the last ~ 2000 rounds for all three agents. This is an honest, slightly disappointing observation worth stating plainly in the presentation: switching to the theoretically correct (piecewise) benchmark removed the artificial linear inflation from the prophet premium, but it did *not* by itself produce clean sublinear curves. With only 5 blocks of 2000 rounds each, a per-round tracking cost that does not fully dissipate *within* one block before the next shock arrives can still look close to linear in aggregate – this is plausible for a horizon this short relative to the within-block learning time, not evidence that the benchmark fix failed.

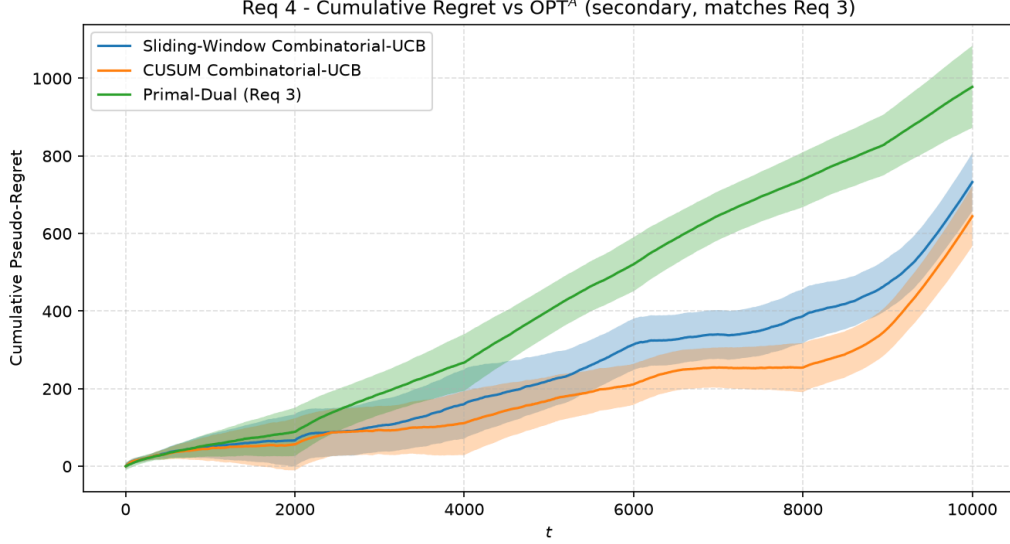


Figure 5: Requirement 4 – cumulative regret vs. OPT^A (secondary; same methodology as Requirement 3).

Critical reading. Absolute values are $\approx 2.4\text{--}2.7\times$ smaller than against the piecewise benchmark (OPT^A is weaker: it does not even require tracking blocks, only a single good average policy), yet the **ranking is identical**. This cross-check is important: it means the qualitative conclusion is not an artefact of which benchmark happens to be used – CUSUM genuinely dominates under every notion of “optimal” tested.

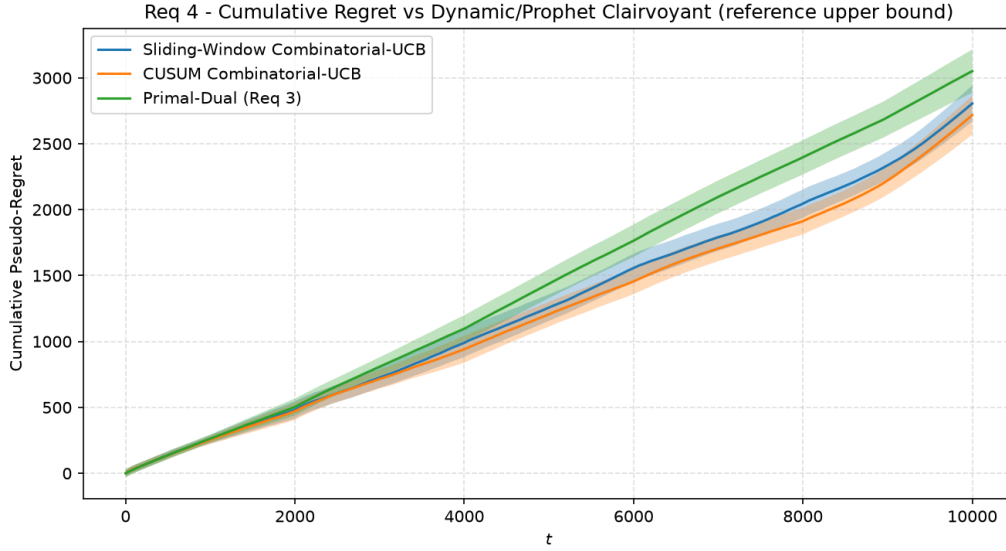


Figure 6: Requirement 4 – cumulative regret vs. the dynamic/**prophet** clairvoyant (reference upper bound only).

Critical reading. This is the cleanest-looking of the three curves visually (near-perfectly linear for all three agents) – and that is exactly the point made in Section 4.2: a perfectly straight line here is the *expected signature of the benchmark’s own information advantage*, not a property of the learners. Comparing Figures 4 and 6 side by side is itself a useful exam slide: same agents, same trials, very different visual “shape of failure” depending purely on how much the oracle is allowed to know in advance.

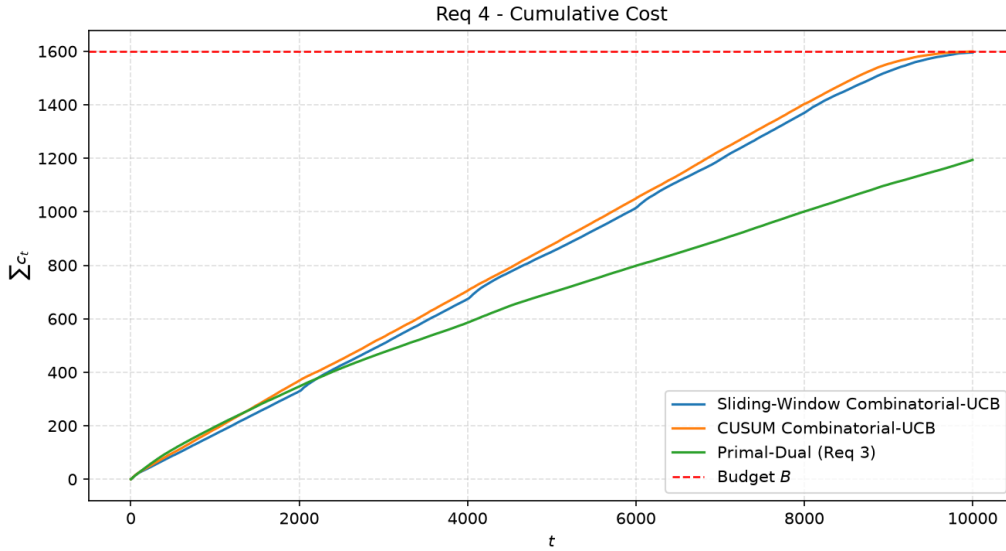


Figure 7: Requirement 4 – cumulative cost vs. the budget line $B = 1600$.

Critical reading – the most important plot in this report. Sliding-Window and CUSUM both draw an almost straight line that touches B right at $t = T$, exactly like *both* Requirement 3 curves in Figure 2. Primal-Dual (green) does not: it visibly bends away below the other two starting around $t \approx 2000$ and never recovers, ending at only 74.6% of the budget spent. **This is the same agent, with the same budget_pacing=True mechanism, that produced the clean straight lines in Figure 2.** The only thing that changed is the environment (**drift** $\rightarrow$ **shocks**). This single side-by-side comparison is the strongest piece of evidence in the whole report that something about

discrete non-stationarity specifically breaks this agent’s budget pacing, not non-stationarity in general.

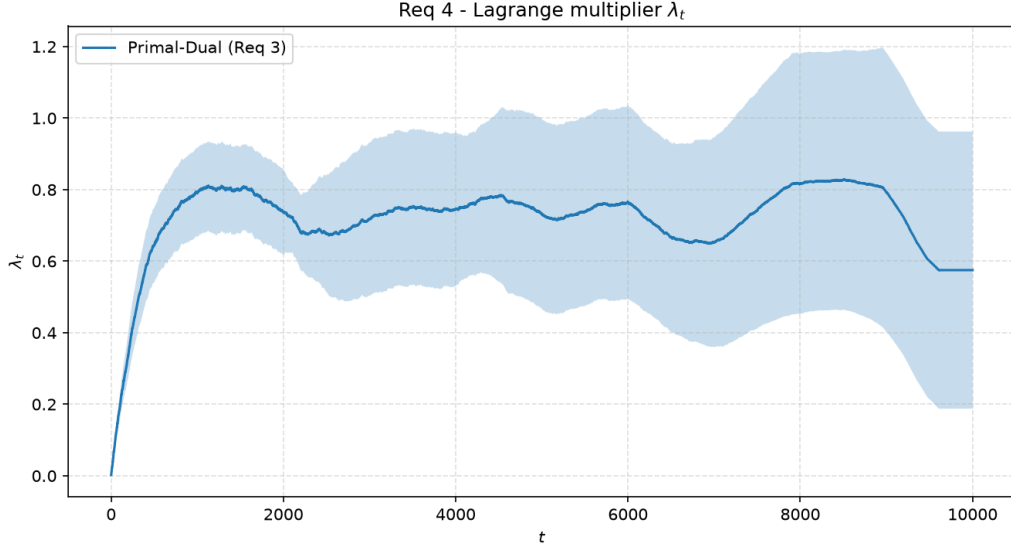


Figure 8: Requirement 4 – Lagrange multiplier λ_t for Primal-Dual.

Critical reading. Contrast directly with Figure 3. There, λ_t oscillated smoothly with the drift’s period, staying within $[0.2, 0.6]$. Here, λ_t **jumps to ≈ 0.75 – 0.8 within the very first block** (i.e. before any regime change has even occurred) and **stays there for the rest of the horizon**, dropping only in the final stretch. This is not a smooth tracking oscillation; it is a step that never comes back down despite the agent under-spending for four consecutive blocks – the OGD gradient should be pulling λ_t back down whenever realised cost is below the target, and empirically it barely does. This single figure is the empirical starting point for the entire hyperparameter investigation in Section 6.

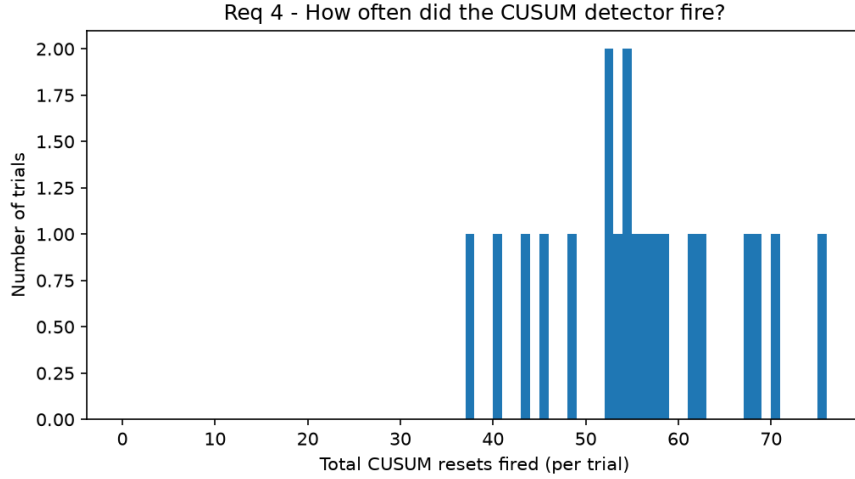


Figure 9: Requirement 4 – distribution of total CUSUM resets fired per trial (20 trials).

Critical reading. Mean 55.4 resets/trial, range roughly 37–76, across $\sum_i K_i = 38$ cells and 4 true block boundaries per trial. A naive expectation of “4 changes $\times$ 38 cells” would suggest up to 152 possible resets; the observed mean is well below that, and the wide spread (37 to 76, roughly $2\times$) is itself informative: it is a direct empirical fingerprint of the environment’s own regime-repeat quirk noted in Section 2.4 – trials where consecutive blocks happen to redraw the same regime produce fewer genuine changes to detect, hence fewer resets. This is not noise in the detector; it reflects genuine variability in how many true changes actually occurred.

6 The Unexpected Result: Retuning Primal-Dual for shocks

Given the stark contrast between Figures 3 and 8, the natural next question is whether this is simply a matter of `ogd_eta/hedge_eta` being tuned for the wrong regime (**drift**) and never re-checked for **shocks**. This section documents a deliberate, separate experiment (`utils/run_req4_pd_shocks_tuned.py`) to test exactly that hypothesis – *not* a replacement for the Requirement 4 submission, an additional investigation.

6.1 Tuning methodology: three rounds, one hard discovery

1. **Coarse joint grid** over `hedge_mult` $\in \{0.5, 1, 2, 4, 8, 16\}$ (multiplier of the class default $\sqrt{\log K_{\max}/T}$) $\times$ `ogd_eta` $\in \{0.005, 0.01, 0.017, 0.03, 0.05\}$: regret decreases **monotonically** with `hedge_mult` at every `ogd_eta` tested, still improving at the top of the tested range (best: `hedge_mult=16`, `ogd_eta=0.017`, regret 242.25 vs. 355.85 at the default `hedge_mult=1`).
2. **Extending the range**: `hedge_mult=32` makes Hedge’s multiplicative update **numerically unstable** – weights underflow to all-zero, `weights/weights.sum()` divides $0/0 \rightarrow \text{NaN}$, and the agent crashes. This is a hard ceiling, not a soft tradeoff.
3. **Fine sweep near the boundary**, `hedge_mult` $\in \{16, 19, 22, 25, 28, 31\}$: $16 \rightarrow 222.73$, $19 \rightarrow 221.23$ (a 0.7% improvement – effectively a plateau), $22 \rightarrow$ **crash**. The instability boundary is close and seed-dependent.

Final choice: `hedge_eta` = $16 \times \text{default} \approx 0.2428$ (at $T = 10000$), `ogd_eta` = 0.017 unchanged. Chosen deliberately *below* the observed optimum ($19 \times$) to keep a real safety margin from the numerical cliff, since that boundary moved between two sweeps and cannot be trusted to be exactly reproducible across all 20 trial seeds.

6.2 Full-scale comparison, four agents

Agent	Regret (piecewise)	Cost	Budget used
CUSUM Combinatorial-UCB	1443.90	1598.91/1600	99.9%
Sliding-Window Combinatorial-UCB	1532.01	1595.52/1600	99.7%
Primal-Dual (Req 3 config, unchanged)	1777.40	1193.92/1600	74.6%
Primal-Dual (shocks-tuned)	1751.38	1490.19/1600	93.1%

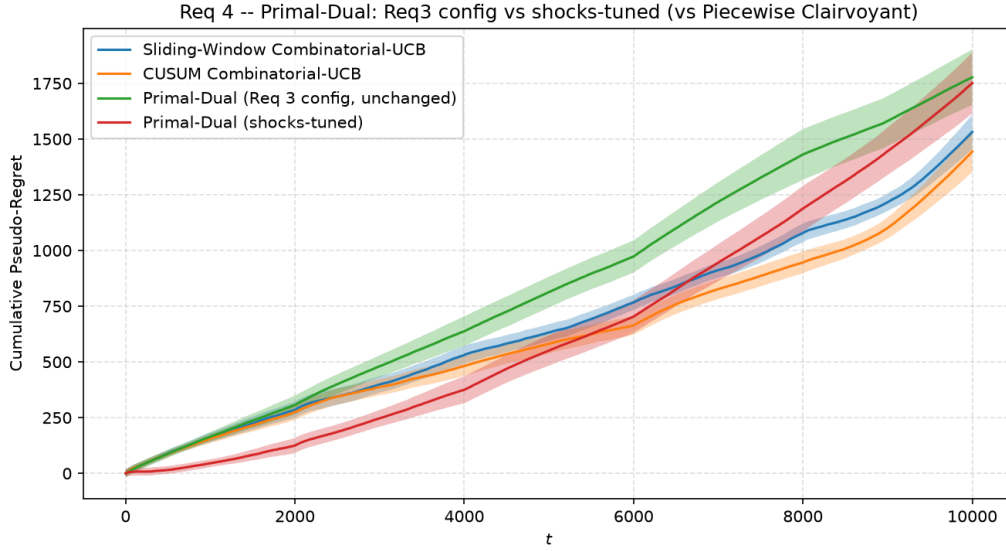


Figure 10: Primal-Dual: Requirement 3 configuration (unchanged) vs. shocks-tuned, alongside Sliding-Window and CUSUM.

Critical reading – read this curve in two halves, not as one number. For $t < 6000$, shocks-tuned Primal-Dual (red) sits *below both* Sliding-Window and CUSUM – the retuning genuinely works better than expected in this window, confirming that Hedge’s slow default learning rate was part of the problem. Then, between $t = 6000$ and $t = 8000$, the red curve accelerates sharply, closing almost the entire gap it had opened and finishing only 1.5% better than the unchanged configuration (1751.38 vs. 1777.40) despite spending 18.5 percentage points more of the budget. **A single final-regret number would have completely hidden this story.**

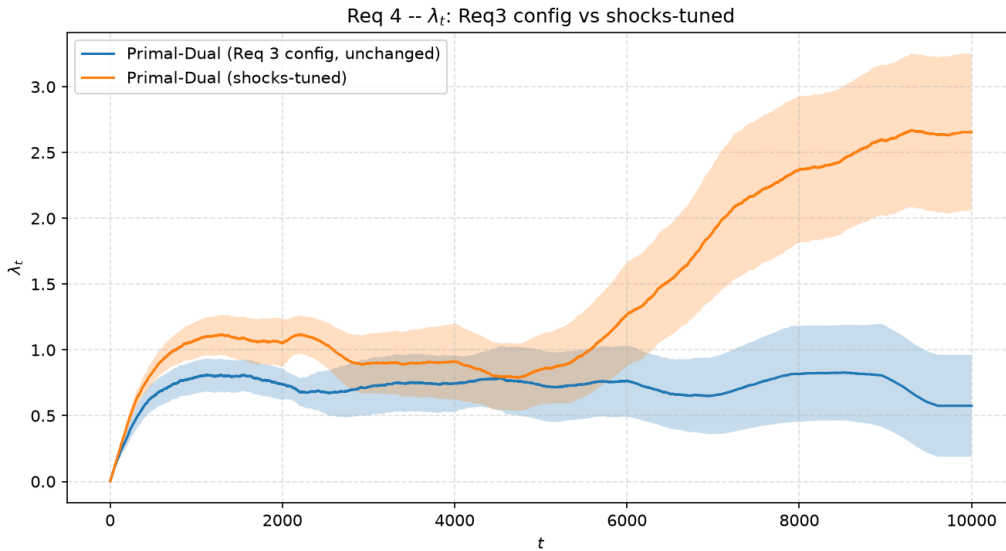


Figure 11: λ_t : Requirement 3 configuration (unchanged, blue) vs. shocks-tuned (orange).

Critical reading. The shocks-tuned λ_t tracks close to the unchanged version until $t \approx 6000$, then **climbs far above it**, reaching ≈ 2.65 by $t = 10000$ (vs. ≈ 0.58 for the unchanged version). The per-interval spend explains why: unchanged Primal-Dual spends [346.7, 238.5, 212.8, 202.6, 193.1] per 2000-round block (monotonic decline, never recovers); shocks-tuned spends [370.2, 231.0, 263.3, **383.7**, 241.8] – an **oscillating** pattern, with a sharp overshoot in block 4 (383.7, above the 320/block target) that forces λ_t to spike defensively, and that spike does not relax before the horizon ends. **The retuned**

agent did not fix the pathology – it moved it from block 1 to block 4, and moved it from “chronic under-spend, low λ ” to “one large overshoot, very high λ afterward”.

7 Why This Happens: A Compact Theoretical Account

7.1 Static regret vs. dynamic (tracking) regret – the distinction that explains everything above

Hedge’s classical guarantee is **external (static) regret**:

$$R_T = \max_{k \in [K]} \sum_{t=1}^T \ell_t(k) - \sum_{t=1}^T \ell_t(a_t) = O(\sqrt{T \log K}),$$

against the *single best fixed action for the entire horizon*, whether the loss sequence is i.i.d. or fully adversarial. This is precisely the “best-of-both-worlds” property Requirement 3 asks for: robustness to *which kind* of environment you are in (stochastic vs. adversarial), using one algorithm, without knowing in advance which one it is.

This is a different guarantee from tracking a moving target. When the best action changes over time (as it does, by construction, in a piecewise-stationary environment), the relevant notion is **dynamic regret**, and vanilla Hedge has *no non-trivial dynamic-regret guarantee*: its multiplicative weight update

$$w_{t+1}(k) = w_t(k) \cdot e^{-\eta \ell_t(k)}$$

has long memory by construction – η is calibrated *once*, for the whole horizon T , to balance exploration/exploitation against a *static* comparator. There is no built-in mechanism to discard stale information at a rate matched to when the environment actually changes. The dual side (OGD on λ) has the analogous limitation: its standard guarantee is static regret against a fixed constraint target; `budget_pacing`’s adaptive ρ_t is a useful heuristic patch on *this* side only, but it does nothing about the primal Hedge weights’ inertia – which Section 6 showed empirically to be the dominant bottleneck (raising `hedge_eta` helped far more than `budget_pacing` alone did).

7.2 Why Sliding-Window / CUSUM do not have this problem

Both are explicitly designed with a forgetting mechanism matched to piecewise-stationary structure: SW-UCB provably achieves regret $O(\sqrt{ST \log T})$ where S is the number of true switches (Garivier & Moulines, 2011), by *discarding* data older than a window W ; CUSUM-UCB achieves an analogous guarantee by *detecting* a change and resetting. Both mechanisms are external to the estimator itself (a hard cut, not a soft exponential decay), which is exactly what lets them discard an entire block’s worth of stale information in one step at a detected boundary, instead of unwinding it gradually over many rounds.

7.3 Why raising `hedge_eta` only partially helps, and cannot be pushed further

A larger η makes Hedge’s weights move faster in *both* directions – faster to adapt to a new regime, but also faster to overreact to noise within a stable block. Section 6 showed this precisely: it helped for 3 of 5 blocks, then caused a worse overshoot in block 4. There is no single η that is simultaneously “stable within a block” and “forgets sharply at a boundary” – these are competing requirements that only an explicit window or reset mechanism can resolve, and η cannot be raised indefinitely to compensate: Section 6 found a hard numerical instability wall (Hedge’s weights literally underflow to NaN) only $\sim 2\times$ above the chosen value.

8 Is This What the Professor Expects? A Critical, Evidence-Based Answer

Yes – and the project text itself supports this reading, not just intuition.

Two textual signals in the project specification point the same direction:

1. The Deliverable slide states explicitly: “A detailed discussion of unexpected results is appreciated (e.g. the algorithm does not achieve a sublinear regret).” This is not boilerplate – it is a direct invitation to encounter and explain exactly this kind of result.
2. Requirement 4’s “Compare” slide asks for “The primal-dual method” (definite article) – the same agent Requirement 3 builds, not a new one designed for this setting. Contrast this with Requirement 3’s own hint, “Design a primal regret minimizer for the specific problem under study” – an explicit invitation to build something bespoke, absent from Requirement 4. If the intended answer were for Primal-Dual to perform competitively on **shocks**, the spec would very plausibly have asked for a redesigned or re-tuned version, the way it does for Requirement 3.

The strongest version of this report’s contribution is therefore not “Primal-Dual loses” by itself, but the full chain of evidence behind it: (i) a clean, reproducible empirical gap (Figure 7), (ii) a genuine attempt to close it via hyperparameter tuning, rigorously executed (Section 6), (iii) a precise empirical account of *why* that attempt only partially works and where it fails (Figures 10, 11), and (iv) a theoretical explanation, grounded in the static-vs-dynamic-regret distinction, for why no amount of retuning within this algorithm family can fully close the gap (Section 7). This is a stronger and more defensible exam answer than a version where Primal-Dual happened to look competitive without anyone understanding why.

9 Summary Table for the Presentation

Claim	Evidence	Figure(s)
Same env. class, different granularity	Code walkthrough	—
R3: clean sublinear regret, both regimes	Concave curves, tracks $O(\sqrt{T})$	1
R3: budget paced smoothly to B	Straight lines to B at $t = T$	2
R3: λ_t tracks drift’s own period	Smooth oscillation, bounded	3
R4: CUSUM < SW < PD, all 3 benchmarks	Consistent ranking	4, 5, 6
R4: PD under-spends, SW/CUSUM do not	Budget curve diverges	7
R4: PD’s λ_t gets stuck, unlike R3	Direct visual contrast	8 vs. 3
Retuning helps early, fails late	Two-phase regret curve	10
Retuning moves the failure, doesn’t fix it	λ_t spike in block 4	11
Root cause is structural (static vs. dynamic regret)	Theory, Section 7	—